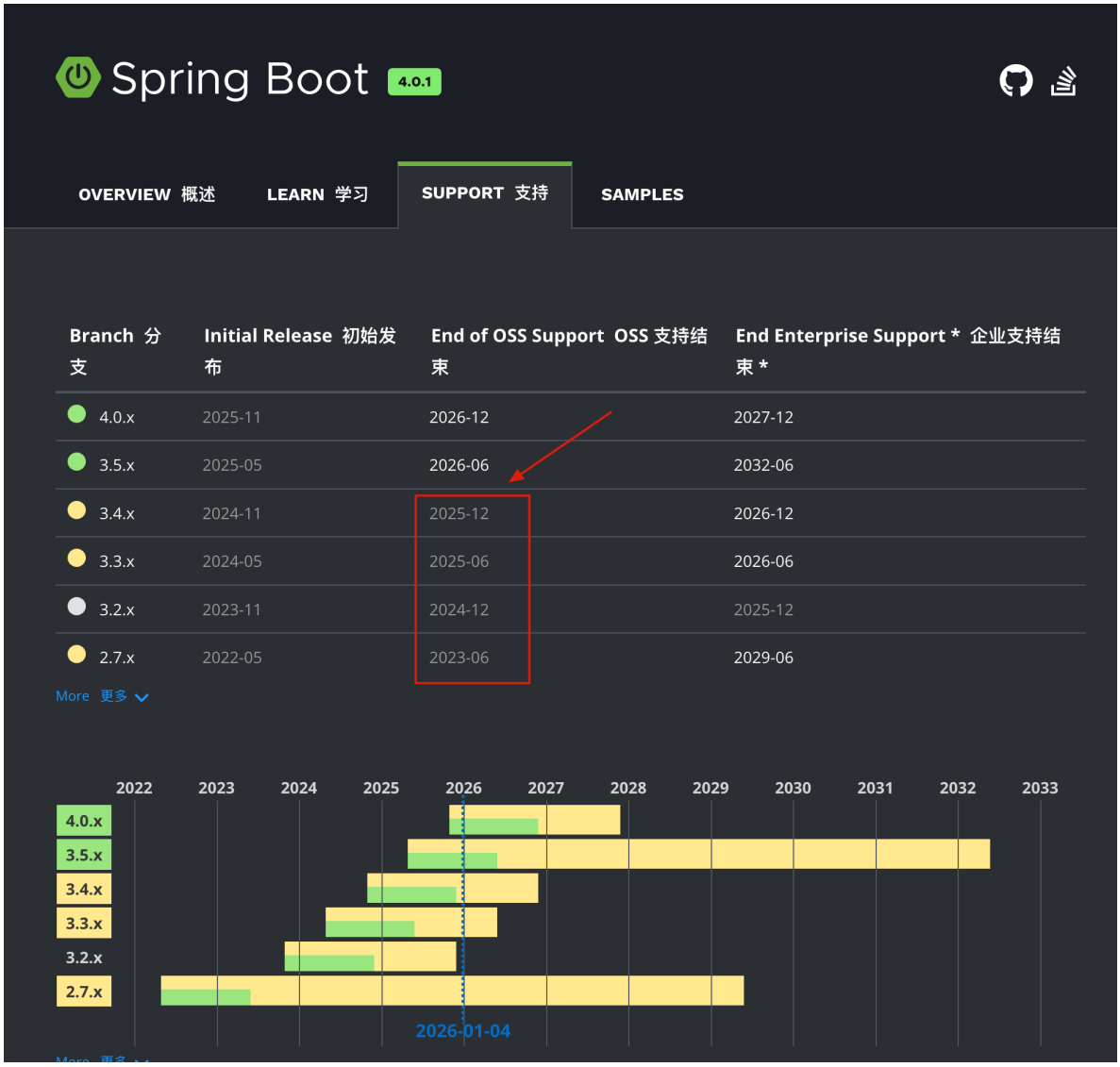
大家好，我是 Guide！技术的迭代速度有时候真的快到让人窒息。

就在前阵子，**Spring Boot 4.0** 正式发布，3.2.x、3.3.x 和 3.4.x 这些 2024 年发布的版本官方已经不在维护了，也就是不再提供免费的安全更新和错误修复。

下图来源于 SpringBoot 官方文档：https://spring.io/projects/spring-boot#support 。



其实影响也不大，很多老项目目前还是 2.x 版本，甚至还有停留在 1.x 版本的。

对于公司项目，如果能稳定运行，就不必要强行去升级新版本。你升级了，老板看不到效果，还容易出现问题，一出现问题说不定就是你背锅。

不过，对于新项目和个人项目来说，还是应该尽量使用相对较新且稳定的版本。

InterviewGuide（基于 Spring Boot 4.0 + Java 21 + Spring AI 开发的 AI 简历分析和模拟面试系统）这个项目我当时图省事复制了上一个项目的部分依赖文件，导致 SpringBoot 版本是 3.3，给自己找了一个麻烦！

有不少朋友反馈建议将其升级为 4.0 版本，于是我昨天就把这件事做了！

这一路踩了不少坑，尤其是 Jackson 3 的包名大改，简直是“史诗级”的 Breaking Change。今天就把这份**穿透式升级指南**分享给大家，建议收藏备用！

具体的更新内容可以查看这个 Commit：https://github.com/Snailclimb/interview-guide/commit/f7bb05980b725c8658f93a4c511bf4b2ec616b82 。

为什么要升级?

Spring Boot 4.0.0 基于 Spring Framework 7.0，全面支持 Java 25（含虚拟线程优化）。核心新特性包括：HTTP Service Clients 简化远程调用；原生 API 版本管理；全面采用 JSpecify 空安全体系（默认非空，编译期防 NPE）；关键依赖升级至 Jackson 3.0、Tomcat 11、Hibernate 7.1 等；模块化设计改进；化支持 Gradle 9；Redis 静态主从配置；移除 Undertow。

我专门分享过一篇文章介绍这些新特性：Spring Boot 4.0 正式发布，人已麻。。。（确实已经麻了）。

版本变更总览

在这里提个醒，升级千万别“跳级”。官方建议的升级路径：**3.3/3.4 → 3.5 → 4.0**，这条路径能避免 90% 的坑。

组件	升级前	升级后
Spring Boot	3.3.6	4.0.1
Spring Framework	6.x	7.x
Spring AI	1.1.2	2.0.0-M1
Redisson	3.24.3	4.0.0
Gradle	8.8	8.14
Hibernate	6.x	7.2.0.Final
Tomcat	10.x	11.0.15
Jackson	2.x	3.x
iText	7.2.5	8.0.5
MapStruct	1.5.5.Final	1.6.3

升级步骤详解

Step 1: 升级 Gradle

Spring Boot 4.0 要求 Gradle 8.14 或更高版本。首先需要升级 Gradle Wrapper：

```
# 临时使用旧版本 Spring Boot 执行 wrapper 升级命令
./gradlew wrapper --gradle-version=8.14
```

修改 `gradle/wrapper/gradle-wrapper.properties`：

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.14-bin.zip
```

Step 2: 更新依赖版本

修改 `gradle/libs.versions.toml`：

```
[versions]
spring-boot = "4.0.1"
spring-ai = "2.0.0-M1"
redisson = "4.0.0"
mapstruct = "1.6.3"
aws-sdk = "2.29.51"
itext = "8.0.5"
lombok = "1.18.36"
junit-jupiter = "5.12.0"

[plugins]
spring-boot = { id = "org.springframework.boot", version.ref = "spring-boot" }
spring-dependency-management = { id = "io.spring.dependency-management", version
= "1.1.7" }
```

`libs.versions.toml` 是 Gradle 7.0 引入并从 7.4 版本开始正式推荐的 **"版本目录" (Version Catalog)** 文件。它的核心作用是**集中管理项目中所有的依赖库及其版本号。**

Step 3: 模块化重构 (Starter 变更)

Spring Boot 4.0 采用了更细粒度的模块化设计，需要更新 starter 依赖：

```
// 升级前
implementation 'org.springframework.boot:spring-boot-starter-web'

// 升级后
implementation 'org.springframework.boot:spring-boot-starter-webmvc'
```

完整的 `build.gradle` 依赖变更：

```
dependencies {
    // Spring Boot Starters (Boot 4.0 modular design)
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    // Spring AI 2.0
    implementation "org.springframework.ai:spring-ai-starter-model-
openai:${libs.versions.spring.ai.get()}"
    implementation "org.springframework.ai:spring-ai-starter-vector-store-
pgvector:${libs.versions.spring.ai.get()}"

    // Redisson 4.0
    implementation "org.redisson:redisson-spring-boot-
starter:${libs.versions.redisson.get()}"

    // iText 8
    implementation "com.itextpdf:itext-core:${libs.versions.itext.get()}"
    implementation "com.itextpdf:font-asian:${libs.versions.itext.get()}"
}
```

Step 4: 史上最坑的 Jackson 3 迁移

这是本次升级最痛苦的部分。Jackson 3 将包名从 `com.fasterxml.jackson` 全面改为了 `tools.jackson`。这意味着你项目中所有的 `ObjectMapper`、`JsonNode` 导入都失效了。

需要修改的导入语句：

升级前	升级后
`com.fasterxml.jackson.databind.ObjectMapper`	`tools.jackson.databind.ObjectMapper`
`com.fasterxml.jackson.core.type.TypeReference`	`tools.jackson.core.type.TypeReference`
`com.fasterxml.jackson.core.JsonProcessingException`	`tools.jackson.core.JacksonException`

示例代码修改：

```java
// 升级前
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;

try {
    String json = objectMapper.writeValueAsString(data);
} catch (JsonProcessingException e) {
    // 处理异常
}

// 升级后
import tools.jackson.core.JacksonException;
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.ObjectMapper;

try {
    String json = objectMapper.writeValueAsString(data);
} catch (JacksonException e) {
    // 处理异常
}
```

我们需要在 `PdfExportService`、`RedisService` 等 7 个核心业务类中，利用 IDE 的全局替换功能，将 `com.fasterxml.jackson` 统一修正为 `tools.jackson`。

再多提一点，升级 Spring Boot 4 后，如果你执行 `mvn dependency:tree` 或者 `./gradlew dependencies`，你会发现一个非常奇葩的现象：Jackson 2 和 3 竟然会同时出现。

```
spring-boot-starter-jackson (4.x)
├── tools.jackson.core:jackson-core:3.x        ← Jackson 核心实现（新包）
└── com.fasterxml.jackson.annotation:jackson-annotations:2.x ← Jackson 注解（旧包）
```

```
+--- commons-logging:commons-logging:1.3.5
\--- org.jspecify:jspecify:1.0.0
\--- org.springframework:spring-context:7.0.2
+--- org.springframework:spring-aop:7.0.2
+--- org.springframework:spring-beans:7.0.2
\--- org.springframework:spring-core:7.0.2 (*)
\--- org.springframework:spring-core:7.0.2 (*)
+--- org.springframework:spring-beans:7.0.2 (*)
+--- org.springframework:spring-core:7.0.2 (*)
+--- org.springframework:spring-expression:7.0.2
\--- org.springframework:spring-core:7.0.2 (*)
\--- io.micrometer:micrometer-observation:1.16.1
+--- org.jspecify:jspecify:1.0.0
\--- io.micrometer:micrometer-commons:1.16.1
\--- org.jspecify:jspecify:1.0.0
+--- jakarta.annotation:jakarta.annotation-api:3.0.0
\--- org.yaml:snakeyaml:2.5
+--- org.springframework.boot:spring-boot-starter-jackson:4.0.1
+--- org.springframework.boot:spring-boot-starter:4.0.1 (*)
\--- org.springframework.boot:spring-boot-jackson:4.0.1
+--- org.springframework.boot:spring-boot:4.0.1 (*)
\--- tools.jackson.core:jackson-databind:3.0.3
+--- com.fasterxml.jackson.core:jackson-annotations:2.20
+--- tools.jackson.core:jackson-core:3.0.3
\--- tools.jackson:jackson-bom:3.0.3
+--- com.fasterxml.jackson.core:jackson-annotations:2.20 (c)
+--- tools.jackson.core:jackson-core:3.0.3 (c)
\--- tools.jackson.core:jackson-databind:3.0.3 (c)
\--- tools.jackson:jackson-bom:3.0.3 (*)
+--- org.springframework.boot:spring-boot-starter-tomcat:4.0.1
+--- org.springframework.boot:spring-boot-starter:4.0.1 (*)
+--- org.springframework.boot:spring-boot-starter-tomcat-runtime:4.0.1
+--- org.springframework.boot:spring-boot-tomcat:4.0.1
```

很多同学第一反应是“依赖冲突了”。其实不然，这是 Jackson 团队一次非常**工程化、现实主义**的设计。

他们意识到整个 Java 生态（数以万计的第三方库）不可能一夜之间全改包名。于是采取了“**核心实现迁新包，注解沿用旧包**”的策略。这意味着你常用的 `@JsonProperty`、`@JsonIgnore` 完全不用动，这种**渐进式迁移**的思路，值得我们学习。

Step 5: Redisson 4.0 API 迁移

Redisson 4.0 对部分类进行了包重组，`StreamMessageId` 等 Stream 相关类移至新包：

```
// 升级前
import org.redisson.api.StreamMessageId;

// 升级后
import org.redisson.api.stream.StreamMessageId;
```

这个还好点，只涉及到 3 个类：

- `infrastructure/redis/RedisService.java`
- `modules/resume/listener/AnalyzeStreamConsumer.java`
- `modules/knowledgebase/listener/VectorizeStreamConsumer.java`

验证升级结果

升级完成后，运行以下命令验证：

```
# 编译项目
./gradlew :app:compileJava

# 运行测试
./gradlew :app:build

# 启动应用
./gradlew :app:bootRun
```

成功启动后，日志应显示：

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.0.1)

... Tomcat initialized with port 8080 (http)
... Starting Servlet engine: [Apache Tomcat/11.0.15]
... Redisson 4.0.0
... HHH000001: Hibernate ORM core version 7.2.0.Final
```

参考资料

- Spring Boot 4.0 发布公告
- Spring Boot 4.0 Migration Guide
- Spring AI 2.0.0-M1 发布公告
- Redisson Changelog
- Jackson 3 Release Notes

总结

本次Spring Boot 4.0 的升级主要涉及以下几个方面：

1. **构建工具升级**：Gradle 8.14+
2. **模块化变更**：`starter-web` → `starter-webmvc`
3. **Jackson 3 迁移**：包名从 `com.fasterxml.jackson` 改为 `tools.jackson`
4. **第三方库兼容**：Redisson、Spring AI 等需要升级到对应的兼容版本

遵循官方推荐的 3.3 → 3.5 → 4.0 升级路径，可以更平滑地完成迁移。升级过程中遇到的大多数问题都与包名变更有关，通过全局搜索替换即可解决。